Close

Krishna Kumar Trivedi
Exp-5

Krishna Kumar Trivedi Exp-5

```
from google.colab import files
uploaded = files.upload()  # select Social Network Ads.csv
```

Choose Files Social_Network_Ads.csv

- **Social_Network_Ads.csv**(text/csv) - 4903 bytes, last modified: 3/3/2026 - 100% done

```
Saving Social_Network_Ads.csv to Social_Network_Ads.csv
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    confusion_matrix, classification_report
)

warnings.filterwarnings("ignore")
```

```
# Load CSV into DataFrame
df = pd.read_csv("Social_Network_Ads.csv")
df.head()
```

	Age	EstimatedSalary	Purchased
0	19	19000	0
1	35	20000	0
2	26	43000	0
3	27	57000	0
4	19	76000	0

Next steps: Generate code with df New interactive sheet

```
# --------------------- BLOCK 3: Explore Dataset ---------------------
print("Shape:", df.shape)
print("Missing values:\n", df.isna().sum())
```

```
Shape: (400, 3)
Missing values:
 Age               0
EstimatedSalary    0
Purchased          0
dtype: int64
```

```
# --------------------- BLOCK 4: Define Features & Target ---------------------
X = df.drop("Purchased", axis=1)   # Features
y = df["Purchased"]                # Target
```

```
# --------------------- BLOCK 5: Train-Test Split ---------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
# --------------------- BLOCK 6: Feature Scaling ---------------------
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# --------------------- BLOCK 7: Train Logistic Regression ---------------------
model = LogisticRegression()
model.fit(X_train_scaled, y_train)
```

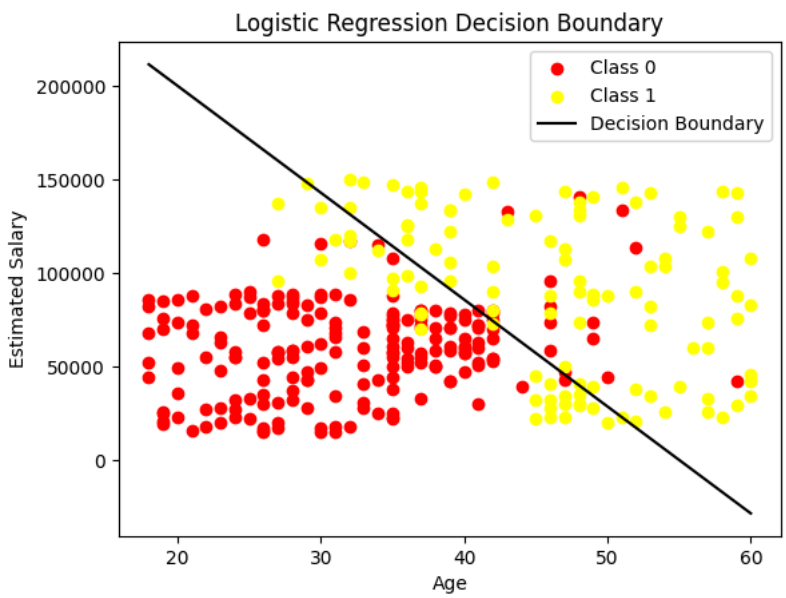
```
# Predictions
y_pred = model.predict(X_test_scaled)
```

```
# --------------------- BLOCK 8: Evaluation Metrics ---------------------
print("Accuracy Score:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Precision Score:", precision_score(y_test, y_pred))
print("Recall Score:", recall_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy Score: 0.8375
Confusion Matrix:
 [[48  3]
 [10 19]]
Precision Score: 0.8636363636363636
Recall Score: 0.6551724137931034

Classification Report:
               precision    recall  f1-score   support

           0       0.83      0.94      0.88        51
           1       0.86      0.66      0.75        29

    accuracy                           0.84        80
   macro avg       0.85      0.80      0.81        80
weighted avg       0.84      0.84      0.83        80
```

```
# Only works for 2 features: Age and EstimatedSalary
w_scaled = model.coef_[0]
b_scaled = model.intercept_[0]

# Convert coefficients to original scale
w_original = w_scaled / scaler.scale_
b_original = b_scaled - np.sum((scaler.mean_ / scaler.scale_) * w_scaled)

# Prepare grid for Age
x_values = np.linspace(X["Age"].min(), X["Age"].max(), 100)
y_values = -(w_original[0] * x_values + b_original) / w_original[1]

# Plot data points
plt.scatter(X_train[y_train==0]["Age"], X_train[y_train==0]["EstimatedSalary"], color="red", label="Class 0")
plt.scatter(X_train[y_train==1]["Age"], X_train[y_train==1]["EstimatedSalary"], color="yellow", label="Class 1")

# Plot decision boundary
plt.plot(x_values, y_values, color="black", label="Decision Boundary")

plt.xlabel("Age")
plt.ylabel("Estimated Salary")
plt.title("Logistic Regression Decision Boundary")
plt.legend()
plt.show()
```